

Lab 2 — Task 8: Containerization and Orchestration Report

Akash Singh Sagar | Repository: github.com/akashsinghsagar/devops-lab2

Objective

Describe the architecture, deployment strategy, scaling results and observations of the Lab 2 experiments, in which a sample Flask application was containerized with Docker, its containers, storage and networking were explored, and it was then deployed, updated, scaled and continuously deployed on Kubernetes (Minikube).

1. Environment and tools

Component	Role in the lab
Docker (Docker Desktop)	Builds images and runs containers; provides layers, volumes, bind mounts and networks.
Docker Compose	Defines and starts the multi-container setup from one file.
Minikube	Single-node local Kubernetes cluster (Docker driver).
kubectl	Command-line client used to apply manifests, inspect, update and scale workloads.
Flask app — lab2-python-app	Sample application listening on port 5000 with a / endpoint and a /health endpoint.
Git, GitHub, GitHub Actions	Source control and the continuous-deployment pipeline.
VS Code, PowerShell	Editor and terminal used for all commands and screenshots.

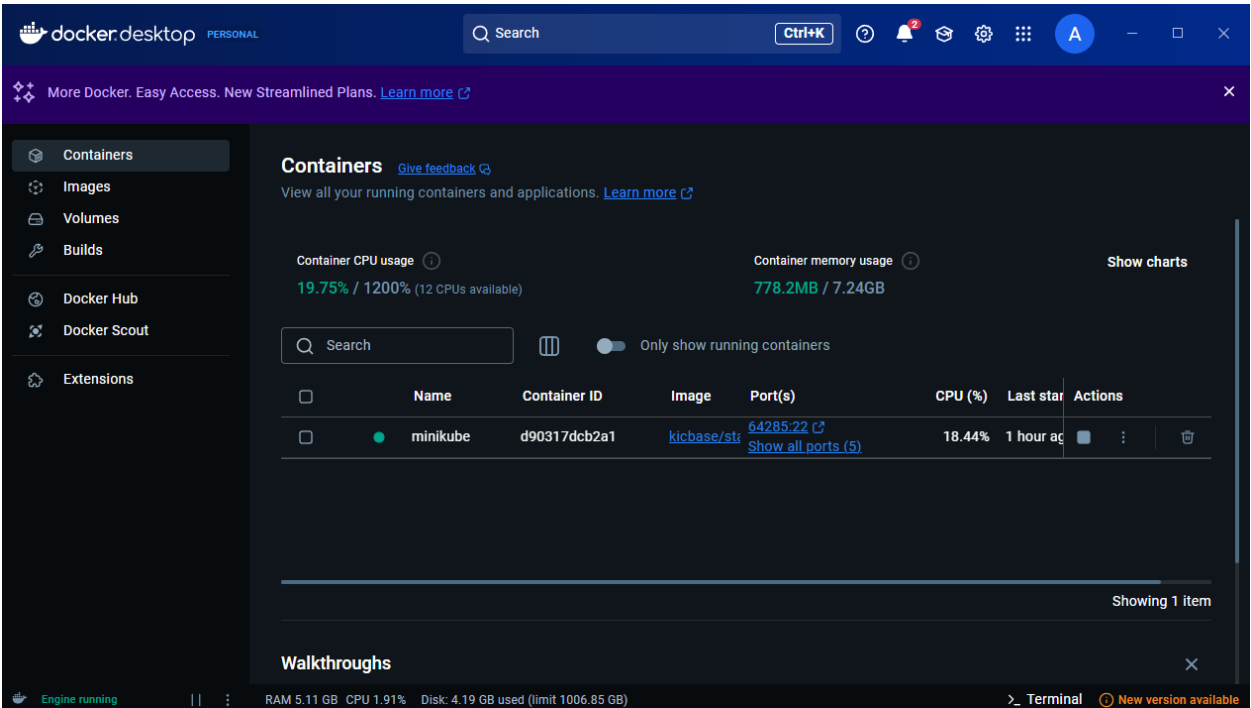


Figure 1 — Tool versions used for the experiments.

2. Architecture

Container layer. The application is packaged as an image built from a Dockerfile; each Dockerfile instruction adds a read-only layer, and a running container adds a thin writable layer on top. Data written to that layer is lost when the container is removed, so persistent data is kept on a Docker named volume or a bind mount. Containers that must talk to each other are attached to a user-defined bridge network, where Docker's embedded DNS resolves container names and no ports need to be published to the host.

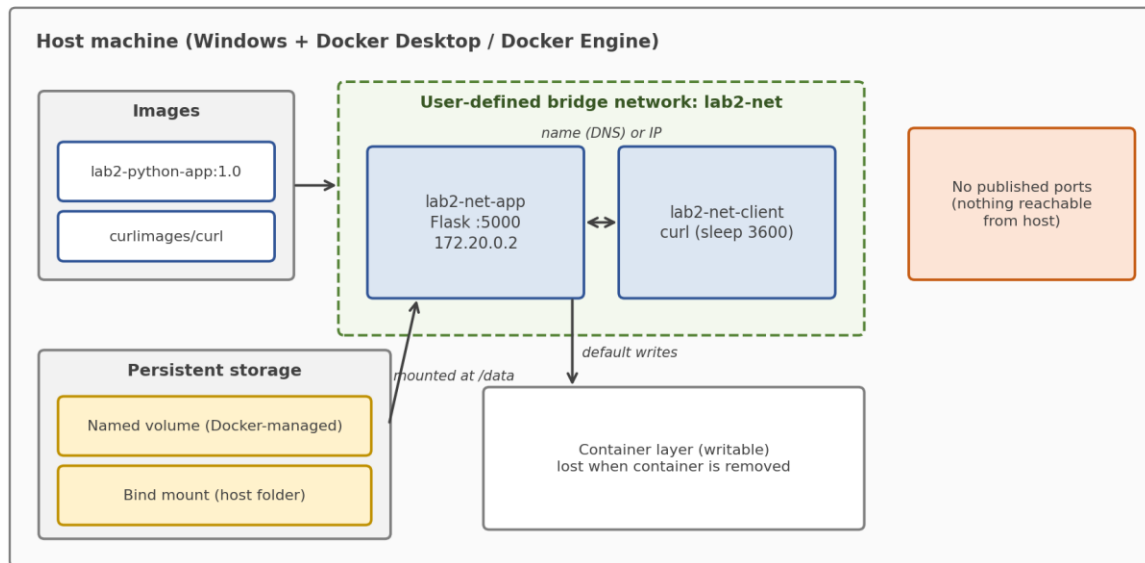


Figure 2 — Docker-level architecture: images, the lab2-net bridge network with two containers, and persistent storage options.

Orchestration layer. On Kubernetes, the same image runs in pods managed by a Deployment. The Deployment owns a ReplicaSet that keeps the desired number of identical pods running; a Service with a stable NodePort address selects the pods by label and distributes requests among the ones that are ready. Readiness and liveness probes on `/health` decide whether a pod receives traffic and whether it is restarted. The manifests are stored in Git and applied automatically by a GitHub Actions pipeline.

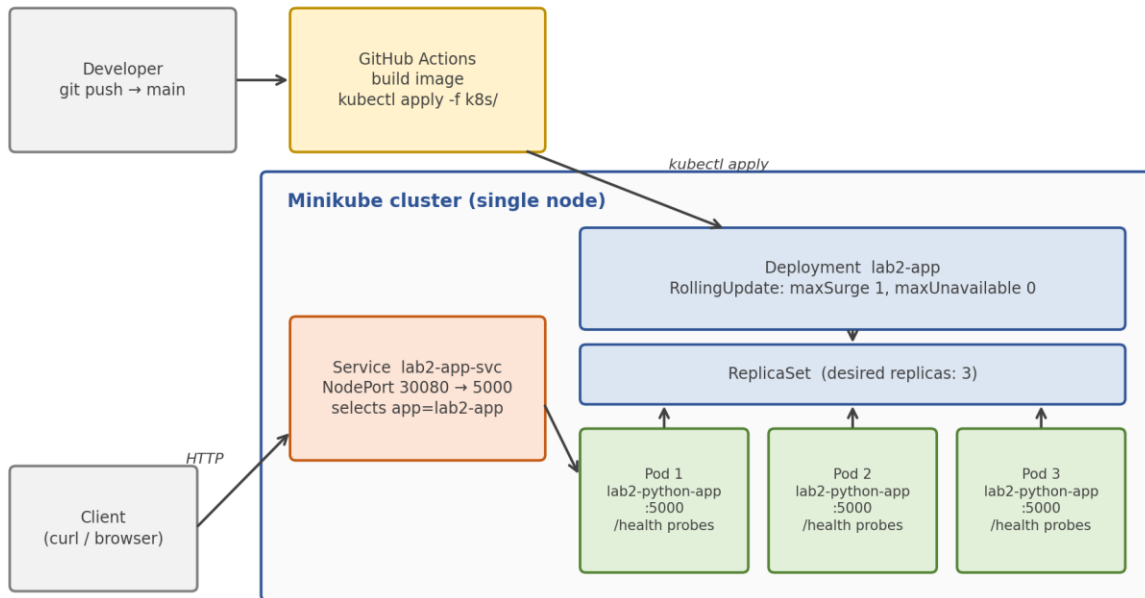


Figure 3 — Kubernetes-level architecture: Service → Deployment → ReplicaSet → pods in Minikube, with manifests applied by the CD pipeline.

3. Containerization summary (Tasks 1–4)

Topic	What was done	Key observation
Image and container	Wrote a Dockerfile, built lab2-python-app:1.0, ran it as a container and documented the layers.	An image is a stack of read-only layers; a container adds a writable layer.
Container and image management	Listed, started, stopped and removed containers; tagged and pushed the image; used Docker Compose.	Compose starts a multi-container setup with one command.
Storage	Compared data in the container layer with a named volume and a bind mount.	Container-layer data disappears with the container; volumes and bind mounts persist.
Networking	Created the user-defined bridge lab2-net and connected the app and a curl client to it.	The app is reachable by container name (embedded DNS) and by IP (172.20.0.2) without any published ports.

4. Deployment strategy

- **Deployment with 3 replicas** — Kubernetes continuously reconciles the cluster to three running pods.
- **RollingUpdate with maxSurge 1 / maxUnavailable 0** — an update adds one new pod at a time and removes an old pod only after its replacement is ready, so capacity never drops below three pods.
- **Readiness and liveness probes** — traffic is only sent to pods whose /health endpoint answers; unresponsive pods are restarted.
- **NodePort Service (30080 → 5000)** — one stable entry point in front of changing pods.
- **Versioned images and revision history** — each release has its own image tag (1.0, 2.0, later the commit SHA), so kubectl rollout undo can return to a known-good revision.

- **Continuous deployment** — a push to main builds the image, updates the manifest with the new tag, runs `kubectl apply` and verifies the rollout.

5. Update and scaling results

Operation	Command	Outcome
Initial deployment	<code>kubectl apply -f k8s/</code>	3/3 pods Running and Ready; the Service lists three endpoints.
Rolling update 1.0 → 2.0	<code>kubectl set image ...:2.0</code>	Pods replaced gradually with no loss of availability; revision 2 recorded in the history.
Failed update (tag 99.0)	<code>kubectl set image ...:99.0</code>	New pod stuck in ImagePullBackOff; existing pods kept serving requests.
Rollback	<code>kubectl rollout undo</code>	Deployment returned to the last good revision; all pods Running.
Scale up	<code>kubectl scale --replicas=5</code>	Two additional pods created; 5/5 ready.
Scale down	<code>kubectl scale --replicas=2</code>	Three pods terminated; 2/2 ready. Deployment later restored to 3 replicas.

```
C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl get deployments,pods,svc
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/lab2-app            3/3      3              3             44m

NAME                                READY    STATUS    RESTARTS    AGE
pod/lab2-app-c95689f9c-hp62k        1/1      Running    0            27m
pod/lab2-app-c95689f9c-qtc1p        1/1      Running    0            21m
pod/lab2-app-c95689f9c-sz74m        1/1      Running    0            26m

NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)        AGE
service/kubernetes                  ClusterIP    10.96.0.1      <none>          443/TCP        66m
service/lab2-app-svc                NodePort     10.108.173.107 <none>          80:30080/TCP   44m
```

Figure 4 — Running state of the application: Deployment 3/3, three Running pods and the NodePort Service.

```
C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl rollout history deployment/lab2-app
deployment.apps/lab2-app
REVISION  CHANGE-CAUSE
1          <none>
3          Update to version 2.0
4          Update to version 2.0
```

Figure 5 — Revision history after the rolling update, failed update and rollback.

```

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl rollout history deployment/lab2-app
deployment.apps/lab2-app
REVISION  CHANGE-CAUSE
1          <none>
3          Update to version 2.0
4          Update to version 2.0

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl scale deployment lab2-app --replicas=5
deployment.apps/lab2-app scaled

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
lab2-app-c95689f9c-cmpjq           1/1     Running   0           26s
lab2-app-c95689f9c-hp62k           1/1     Running   0           29m
lab2-app-c95689f9c-qtc1p           1/1     Running   0           23m
lab2-app-c95689f9c-sz74m           1/1     Running   0           28m
lab2-app-c95689f9c-v4hvm           1/1     Running   0           26s

```

Figure 6 — Five pods Running after scaling the Deployment to 5 replicas.

```

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl scale deployment lab2-app --replicas=3
deployment.apps/lab2-app scaled

```

Figure 7 — CD pipeline run that builds, applies the manifests and verifies the rollout.

6. Observations

- On a **user-defined bridge network** containers find each other by name through Docker's embedded DNS; the lab therefore needed no published ports for container-to-container traffic.
- **Container-layer data is ephemeral.** Anything that must survive container deletion has to live on a volume or bind mount.
- Kubernetes works from **desired state**: changing replicas or the image in the Deployment is enough, and the controllers create, replace or remove pods to match.
- **maxUnavailable: 0 protects availability.** The deliberately broken 99.0 update never removed a healthy pod, and rollback took a single command.
- Locally built images must be **loaded into Minikube** (minikube image load) and used with imagePullPolicy: IfNotPresent; otherwise Kubernetes tries to pull them from a registry.
- With the Docker driver on Windows the node IP is not reachable from the host, so access goes through **minikube service** (or kubectl port-forward).
- **Unique image tags** (version numbers, commit SHAs) make updates and rollbacks traceable; the mutable latest tag is avoided.
- Keeping manifests in Git and applying them from a pipeline makes deployments **repeatable and auditable**.

7. Docker compared with Kubernetes

Aspect	Docker (single host)	Kubernetes (Minikube)
Running the app	<code>docker run</code> per container	Deployment keeps N pods running
Scaling	Start more containers manually	<code>kubectl scale</code> changes the replica count
Failure handling	Restart policy on one container	Controllers replace failed pods; probes gate traffic

Aspect	Docker (single host)	Kubernetes (Minikube)
Updating	Stop and replace containers	Rolling update with revision history and rollback
Service discovery	Embedded DNS on a user-defined network	Service with stable address and cluster DNS
Load balancing	None built in	Service distributes requests across ready pods

Conclusion

Docker made the application portable and gave it isolated storage and networking; Kubernetes added declarative deployment, self-healing, zero-downtime updates with rollback, easy scaling and a stable Service address; and the CD pipeline removed manual steps from releases. Together they cover the full path from a Dockerfile to an automatically deployed, scalable application.